

LECCIONES APRENDIDAS SOBRE EL PROYECTO DE SOFTWARE

Integrantes:

Marco Giroto, C.I. 28322119

Eduardo Rojas, C.I. 31127326

Manuel Martínez, C.I. 30845197

Ricardo Mejia, C.I. 30886262

Alberto Rodríguez, C.I. 30967762



LECCIONES APRENDIDAS SOBRE EL PROYECTO DE SOFTWARE

I. Datos generales del Proyecto

Acta de cierre del Proyecto de Software	
Proyecto:	Sistema de Gestión Logística (SGL) AgroMarket
Fecha de Inicio del Proyecto:	12 de Octubre de 2025
Cliente:	AgroMarket S.A
Líder del Proyecto:	Manuel Martinez
Miembros del equipo del Proyecto:	Ricardo Andrés Mejia Azuaje Alberto Alejandro Rodríguez Villanueva Eduardo Vicente Rojas Arduini Marco Girotto Vila

II. Lecciones aprendidas sobre la formulación del proyecto de software

En la formulación de nuestro proyecto, Sistema de Gestión Logística para el cliente AgroMarket, logramos encontrar un punto de inflexión en nuestra formación profesional gracias a la creación del Acta de Constitución; este proceso nos enseñó que la formulación es clave y esencial para delimitar estratégicamente la realidad del proyecto y así lograr protegerlo. Gracias al Acta de Constitución logramos entender que un proyecto va mucho más allá de un simple código y cumplir con ciertas funcionalidades, un proyecto de software, sin importar su nivel de complejidad, siempre tendrá que ser fundamentado por una buena planificación que nos permita anticipar cómo será la organización, estructura y flujo de vida de nuestro proyecto.

En primer lugar, como grupo logramos identificar que el alcance del proyecto es el principal mecanismo de defensa del mismo, al redactar y crear el Acta nos enfrentamos al desafío de traducir las necesidades, los procesos manuales y descentralizados del cliente en requisitos tangibles y logrables; comprendimos gracias a esto que existe un riesgo real y silencioso como lo es el “Scope Creep” si no logramos establecer límites y fronteras claras. Debido a esto abandonamos nuestra idea anterior de que para realizar un buen producto solo es necesario saber que programar a nivel técnico, y entendemos ahora que debemos tener los pies en la tierra y saber que hay mucha información de valor que debemos tomar en cuenta gracias al Acta de Constitución, que nos da los cimientos y sobretodo una gran oportunidad para organizar el trabajo a realizar, debido a que si bien nos define qué debemos hacer, también nos indica que no debemos hacer y eso es de gran utilidad.

En segundo lugar, la formulación nos dirigió a tener una visión del panorama completo, y sobretodo nos abrió los ojos ante la realidad financiera y operativa de lo que conlleva desarrollar un proyecto; la realidad aquí es que no solo lleva consigo un costo las horas de

programación, sino que hay otros factores relevantes al momento de contemplar financieramente el proyecto, como lo son los costos operativos, que pese a ser bastante olvidados son fundamentales para un correcto desempeño y sostenibilidad, como las licencias de los distintos programas y servicios que planteamos utilizar, también costos de mantenimiento, reservas de contingencia e incluso la planificación lleva consigo un costo que no muchos están al pendiente solo por menospreciar esta etapa, error que cometimos anteriormente. De todo esto hay que estar al tanto debido a que una mala estimación financiera podría detener el proyecto por un tiempo, o incluso permanentemente, aprendimos que debemos ser rigurosos, responsables y sobre todo realistas con este tema.

En tercer lugar, la formulación nos hizo comprender como equipo que hay que dejar de pensar superficialmente y sólo fijarnos en los riesgos técnicos, porque si bien constituyen la una gran parte a nivel lógico y funcional, detectamos que es un verdadero peligro no considerar la integridad de los datos previos, por eso concluimos que identificar el riesgo de errores en la migración de datos históricos es fundamental al comienzo de cualquier proyecto, no debemos solo centrarnos en innovar y cumplir con las necesidades, sino que debemos tener en cuenta el legado de la empresa y aprender a ser consultores antes que desarrolladores, para así lograr analizar el estado de la información y proponer soluciones lógicas y contundentes.

Por último, creemos que esto nos enseno que todo debe tener un por qué, no se trata de desarrollar un sistema solo para que funcione y cumpla, sino de justificar como y por que esas herramientas resuelven los problemas presentados, la formulación nos enseno a comunicarnos con el cliente en un lenguaje entendible y entender que no importa cuan avanzado sea nuestro código, tecnologías o soluciones sino tiene un impacto tangible en la rentabilidad y eficiencia operativa del negocio. Entendimos que el éxito del proyecto no se mide por la sofisticación del software, sino por su capacidad para transformar una necesidad operativa en una ventaja competitiva real y presentar una solución efectiva.

III. Lecciones aprendidas sobre la gestión de alcance, tiempo, costos y calidad del proyecto de software

En nuestro proyecto pudimos ejemplificar cómo estos factores se deben tomar en cuenta para cumplir con el desarrollo de manera éxitos, cumpliendo con nuestras limitaciones establecidas, siendo estas las fechas de entrega, el presupuesto establecido, y los requisitos funcionales y no funcionales.

En primer lugar, nos topamos con la necesidad de declarar un alcance. Para esto, se tomó en cuenta el enunciado del proyecto, que vendría a representar el contexto en el que desarrollamos el software. De el contexto otorgado, podemos extraer los requerimientos funcionales, es decir, las capacidades que debe tener el software para cumplir con las necesidades de nuestros clientes.

Adicionalmente, se consideró la calidad que debe tener el software desarrollado. Los atributos que deseamos para conseguir la calidad deseada, también llamados requisitos no funcionales, son definidos y se buscan asegurar, algunos mediante la integración de cierto personal con roles definidos, como por ejemplo un especialista en seguridad o un arquitecto de software para asegurar la seguridad o la eficiencia de nuestro sistema respectivamente.

Por otra parte, también se evaluaron los posibles costos en un proyecto. Estos varían desde el personal mencionado anteriormente, todos con distintos roles que pueden exigir más o menos remuneración, pasando por servicios de software que serán usados, como plataformas donde se despliegan otros elementos o suscripciones a inteligencias artificiales, hasta incluso equipamiento fijo, como hardware de computadora, elementos de oficina o incluso renta de lugares de trabajo.

Finalmente, tomando el tema de la gestión del tiempo, podemos ver como este se puede trabajar de distintas maneras y existen varias filosofías, siendo la más popular el enfoque ágil, donde dividimos el tiempo de desarrollo en sprints, donde realizamos una cantidad de tareas y usualmente dura alrededor de dos semanas, y al final de cada una, se evalúa el rendimiento y se pueden tomar medidas para completar las tareas que no se lograron realizar por algún motivo.

IV. Lecciones aprendidas sobre la gestión de recursos humanos del proyecto de software

Partiendo de una comprensión inicialmente superficial de la estructura de AgroMarket S.A., la lección más significativa fue aprender a trascender la teoría genérica del PMBOK para contextualizar los roles en una realidad operativa manual.

Descubrimos que asignar perfiles técnicos sin considerar la fuerte resistencia al cambio del personal de almacén convertía nuestro plan de recursos en un documento inerte; entendimos que el Analista de Requisitos y el Scrum Master no solo debían gestionar tareas, sino actuar como agentes de cambio que tradujeran la incertidumbre del cliente en especificaciones funcionales precisas. Esta transición nos enseñó que la gestión de recursos humanos en software es fallida si se ignora el factor cultural de la empresa, pues un equipo técnicamente brillante es inútil si no logra empatizar con el usuario final que aún depende de procesos basados en hojas de cálculo.

Asimismo, la ejecución del proyecto reveló que la falta de un enfoque técnico preciso en la coordinación del equipo deriva en silos de información que comprometen la integridad del producto final. Aprendimos que la gestión de recursos no termina en la definición de una Matriz RACI, sino en asegurar que las interdependencias técnicas entre el Arquitecto de Software, el Administrador de Datos y el Analista de Seguridad fluyan sin interrupciones.

Reflexionamos críticamente sobre cómo la descoordinación entre el desarrollo backend y frontend no fue solo un fallo de código, sino una carencia en nuestra dirección humana para fomentar la transparencia; hoy comprendemos que gestionar humanos en tecnología implica orquestar una comunicación técnica síncrona mediante herramientas como Jira para garantizar que cada incremento del SGL esté alineado con la visión global del sistema.

Finalmente, la experiencia con la estimación *Bottom-Up* nos permitió valorar el esfuerzo real detrás de cada paquete de trabajo, entendiendo que el tiempo del personal del cliente es un recurso tan crítico como el de nuestros desarrolladores. Subestimar la dedicación requerida para las pruebas de aceptación y la migración de datos nos mostró que una planificación de recursos exitosa requiere un análisis profundo de la capacidad operativa de todos los involucrados.

En conclusión, este proyecto nos transformó de gestores teóricos a líderes conscientes de que el éxito del software reside en un equipo humano que, más allá de cumplir un rol, se sincroniza técnicamente para resolver ineficiencias críticas de negocio, convirtiendo la tecnología en un facilitador estratégico y no en un obstáculo adicional para la organización.

V. Lecciones aprendidas sobre la gestión del riesgo del proyecto de software

En las actividades realizadas en la realización de este proyecto se pudieron aplicar conocimientos prácticos sobre los riesgos que se pueden encontrar en un proyecto de software como el realizado.

En primera instancia, se pudieron identificar distintos posibles riesgos que se tienen al trabajar en proyectos de software, y de distintas naturalezas y fuentes, es decir, muchas cosas que pueden salir mal, desde fallas en el software, como caída en uno de los servicios en los que dependemos, o fallas en el hardware, como que nos topemos con una infraestructura insuficiente para cumplir con nuestros requisitos, hasta cambios completamente mayores a nosotros, como cambios en la empresa o reglamentos nacionales e internacionales.

Adicionalmente, se realizó una estimación de la probabilidad asociada en cada uno de estos riesgos. Por ejemplo, un servicio donde hosteamos nuestro software suelen estar respaldados por grandes empresas, que garantizan redundancia y un alto nivel de confiabilidad, indicando riesgos bajos, mientras que por ejemplo, una falla en la integración de sistemas ya usados por el cliente puede ser más alta, pues es posible que existan tecnologías que sean obsoletas o que ya no sean mantenidas.

De manera similar, se evaluó qué tan peligrosos son los riesgos, y cómo lidiamos con estos. Se podría pensar en común la dificultad en la integración de un sistema a los usuarios pensados puede significar un riesgo no muy catastrófico, que se puede solucionar mediante la promoción, creación de recursos o mejoras en la UX, mientras que una infraestructura insuficiente puede ser más peligrosa, que se puede solucionar mayormente realizando inversiones imprevistas para mejorarla.

VI. Lecciones aprendidas sobre la gestión de las comunicaciones del proyecto de software

En la gestión de las comunicaciones de nuestro proyecto, Sistema de Gestión Logística para AgroMarket, logramos identificar un pilar fundamental para el éxito que muchas veces pasa desapercibido; este proceso nos enseñó que la comunicación no es un simple intercambio de

mensajes, sino el eje articulador que determina la viabilidad de la transición digital. Gracias a la planificación de esta área logramos entender que la comunicación efectiva va mucho más allá de enviar correos o tener reuniones; un proyecto de software, especialmente cuando el cliente viene de procesos manuales, requiere una estrategia de "pedagogía tecnológica" que nos permita alinear las expectativas técnicas con la realidad del negocio y evitar vacíos de información que generen desconfianza.

En primer lugar, como grupo logramos identificar que el equipo de desarrollo debe actuar como un "puente traductor", al analizar a los interesados nos enfrentamos al desafío de traducir conceptos abstractos como APIs o bases de datos a un lenguaje que el cliente entienda en términos de rentabilidad y operatividad; comprendimos gracias a esto que existe una brecha peligrosa entre lo que nosotros decimos y lo que el cliente interpreta. Debido a esto abandonamos nuestra postura de hablar únicamente con tecnicismos, y entendemos ahora que debemos tener la empatía suficiente para explicar cómo cada avance técnico reduce la carga operativa de sus empleados, transformando la comunicación en una herramienta de gestión de expectativas y no solo de reporte de tareas.

En segundo lugar, la planificación nos dirigió a valorar la trazabilidad y la centralización de la información, y sobre todo nos abrió los ojos ante la peligrosidad de la informalidad en los canales de comunicación; la realidad aquí es que un mensaje perdido en un chat privado puede derivar en un error costoso de desarrollo, por lo que herramientas como Jira y Slack, que planteamos en nuestra gestión de recursos, no son un capricho, sino mecanismos de gobernanza. Aprendimos que si una decisión o cambio no está documentado en la plataforma oficial, prácticamente no existe para el proyecto; de todo esto hay que estar al tanto debido a que la memoria es frágil y confiar en ella es un error que no podemos permitirnos, entendiendo que la estructura comunicacional es lo que protege al equipo ante futuros reclamos o malentendidos.

En tercer lugar, la gestión de comunicaciones nos hizo comprender como equipo que hay que dejar de tener miedo a las "malas noticias" y entender la transparencia como un activo, porque si bien es tentador reportar solo los éxitos, detectamos que es un verdadero peligro ocultar retrasos o riesgos por temor a la reacción del cliente. Concluimos que establecer una comunicación tipo "Push" para las alertas tempranas es fundamental; no debemos solo centrarnos en decir "todo va bien", sino que debemos tener la madurez profesional para levantar la mano a tiempo cuando algo se desvía del plan, permitiendo así tomar decisiones correctivas basadas en la realidad y no en ilusiones optimistas.

Por último, creemos que esto nos enseñó que la confianza se construye con claridad, no se trata de abrumar al cliente con datos, sino de entregar la información correcta, a la persona correcta, en el momento adecuado. La gestión de comunicaciones nos enseñó a escuchar activamente lo que el cliente *no* dice pero necesita, y a entender que no importa cuán robusto sea nuestro sistema, si fallamos en comunicar su valor o en gestionar el cambio cultural que implica su uso, el proyecto fracasará. Entendimos que la calidad de un líder y su equipo se mide por su capacidad de mantener a todos los actores alineados bajo una misma visión, transformando la incertidumbre natural del desarrollo en certezas operativas.

VII. Lecciones aprendidas sobre la implementación, cierre y evaluación del proyecto de software

Estrategia de implementación	
Aplicación de la solución	El trabajo se divide por funcionalidad y características entre el equipo de desarrollo supervisado por el líder del proyecto en conjunto, llevando a cabo reportes a la gerencia del proyecto de acuerdo al cronograma de trabajo previamente establecido para mantener un orden. Se ejecutará un despliegue gradual por módulos (Primero se implementa el módulo de inventario, luego de pedidos, clientes, reportes, etc...conforme a lo establecido en la entrega 2). Se iniciará con una fase piloto por cada módulo, permitiendo que los registros manuales y los nuevos coexistan para verificar la integridad de los resultados operativos antes del apagado del sistema antiguo. Una vez desarrollada la solución y probada de forma modular, implementando cada módulo posteriormente al sistema, se procede a la capacitación del personal para manejar el nuevo sistema para comenzar su uso.
Adaptación al entorno	El software se ajustará a la infraestructura actual de AgroMarket mediante la configuración de APIs de bajo consumo. La tarea crítica será la migración automatizada de datos históricos desde Excel, utilizando scripts de validación para asegurar que no existan inconsistencias en el stock al momento del lanzamiento
Garantía de la calidad	Se aplicarán Pruebas de Aceptación de Usuario basadas en los prototipos validados previamente. Se enfocará en la prevención (detección temprana de defectos) sobre la inspección, incluyendo pruebas de simulación (sobre todo en el caso de la base de datos). Se utilizará el modelo ágil para desarrollo con ciclos cortos de entrega y retroalimentación constante del cliente.
Minimización de riesgos	Para mitigar la resistencia al cambio del personal de almacén, se implementará una estrategia de "pedagogía tecnológica" que incluye capacitaciones presenciales y soporte técnico de respuesta inmediata. Además, se contará con un plan de contingencia que permita regresar a los registros manuales. Se tendrá en cuenta la escalabilidad acorde al volumen de transacciones esperadas y se pondrá a prueba el sistema por medio de prueba de estrés para simular el comportamiento en la práctica. Para minimizar los fallos de arquitectura, se implementa una Arquitectura

	Desacoplada, es decir, se busca descentralizar las dependencias (por ejemplo, usar varios equipos para manejo de datos en caso de que las transacciones sean demasiadas en algún caso excepcional). Mantener el sistema actualizado para que cumpla siempre con los requisitos del negocio, para cubrir posible expansión de negocio y por ende mayor flujo o añadir nuevos requerimientos a raíz de mejorar la eficiencia
--	--

[Revisión postmortem del proyecto de software implementado, evaluación de la actuación individual, auditoría del proyecto y lecciones aprendidas del equipo de trabajo del proyecto.]

Evaluación del Proyecto de Software			
Análisis postmortem	Mejoras del proceso	Evaluación de la actuación individual	Auditoría del proyecto
[Identificar los aspectos positivos y/o negativos que se sucedieron en el proyecto de software cuando finalizó cada iteración o sprint.]	[Indicar las mejoras a efectuar en los procesos y procedimientos para que sean consideradas en proyectos futuros.]	[Identificar qué roles estuvieron involucrados en cada uno de los aspectos analizados.]	[Descripción de los controles de calidad aplicados al software desarrollado en cada uno de los aspectos mencionados en el análisis postmortem, con base en los estándares de calidad de software utilizados en el proyecto.]
<u>Requerimientos y Diseño (Nov 2025)</u> Positivo: Validación temprana: Uso de prototipos Negativo: Resistencia inicial: apatía o miedo al sistema por parte del personal operativo	El uso de prototipos en Figma para la interfaz, permitió realizar Revisiones técnicas formales. Esto evitó desarrollar funcionalidades que el personal de almacén no entendería, ahorrando retrabajo futuro. Realizar dinámicas de	Analista de Requisitos / Diseñador UX: Líderes en la mediación entre el negocio y el equipo técnico.	Revisiones Técnicas Formales (RTF): Inspección de los prototipos contra el alcance inicial definido en el Acta de Constitución.

	"User Research" previas para romper la barrera tecnológica. Evitar rediseños tardíos validando requisitos desde el día 1.		
<u>Diseño de la Solución (dic 25)</u> positivo: Definición de arquitectura modular robusta. negativo: Subestimación de la complejidad técnica para integrar el hardware antiguo de la empresa.	Incluir un hito de "Auditoría de Infraestructura" en la fase de diseño para no diseñar sobre supuestos de hardware ideales.	Arquitecto de Software: Encargado de alinear la tecnología web con los recursos de las PC del cliente.	Revisión de Arquitectura: Validación de diagramas de entidad-relación y flujo de datos bajo estándares de seguridad ISO/IEC 25010.
<u>Desarrollo y Pruebas del Módulo de Inventario (enero 26)</u> Positivo: Alta Calidad Técnica Negativo: Falta de sincronización inicial entre	Implementar reuniones diarias (Daily Stand-ups) más rigurosas. La participación activa del líder es vital para mantener la visión global del módulo.	Desarrolladores y Scrum Master: Responsables de la ejecución técnica y de eliminar impedimentos comunicacionales.	Pruebas Unitarias: Cumplimiento estricto del 80% de cobertura. Análisis SAST: Escaneo estático de código para prevenir inyecciones SQL.

Front/Back por comunicación asíncrona ineficiente.			
<u>Desarrollo y Pruebas del Módulo Gestión de Pedidos (marzo 26)</u> positivo: Integración fluida con los datos del inventario. negativo: Aparición de errores de concurrencia al simular múltiples pedidos simultáneos.	Automatizar pruebas de integración desde el inicio del módulo para detectar conflictos de bases de datos antes de las demos.	Desarrolladores y Especialista QA: Encargados de la lógica de negocio y la validación de integridad de las transacciones.	Pruebas de Carga: Evaluación del rendimiento del servidor bajo estrés. Pruebas dinámicas de seguridad en el entorno de desarrollo
<u>Despliegue en Ambiente de Pruebas, Migración de Datos y Capacitación. (abril 26)</u> Positivo: Aceptación positiva tras ver la eficiencia del sistema vs el manual. Negativo:	Iniciar la limpieza y curaduría de los datos históricos del cliente dos meses antes de la migración técnica.	Líder de Proyecto y Especialista en Datos: Responsables de la transición del sistema antiguo al nuevo SGL.	Pruebas de Aceptación de Usuario: Validación final del cliente sobre el entorno de pruebas. Auditoría de integridad de base de datos post-migración.

La limpieza de datos en Excel retrasó el despliegue			
<u>Puesta en Producción del Sistema Web.</u> (24 abril 26) positivo: Estabilidad del sistema en el despliegue negativo: Saturación del soporte técnico por consultas operativas menores ("curva de aprendizaje").	Crear un centro de ayuda (Wiki/Manual) digital integrado en el software para que el usuario sea más autónomo.	DevOps / Equipo de Soporte: Monitoreo activo de la plataforma en el entorno real de AgroMarket.	Monitoreo de Rendimiento: Auditoría de tiempos de respuesta en producción. Verificación de certificados SSL y seguridad en la nube.
<u>Cierre Administrativo y Finalización del Proyecto.</u> (30 abril 26) positivo: Firma de aceptación sin objeciones. negativo: La documentación técnica final se completó apresuradamente por el soporte intensivo.	Designar a un "Documentador" dedicado en cada Sprint para que al cierre administrativo solo se compile la información ya existente.	Líder de Proyecto y Patrocinador: Encargados de la firma del Acta de Cierre y la liberación de recursos del equipo.	Auditoría de Entregables: Verificación de que todos los manuales, códigos y licencias fueron transferidos formalmente al cliente.

VIII. Conclusiones finales del proyecto de software realizado

Como equipo de trabajo, la ejecución del proyecto para el desarrollo del Sistema de Gestión Logística (SGL) de AgroMarket representó un desafío de integración entre la teoría de la gestión de proyectos y la realidad práctica de un desarrollo de software. A continuación, detallamos nuestra retrospectiva:

¿Qué se hizo bien?

A pesar de las dificultades iniciales, se logró establecer una estructura organizativa sólida, consiguiendo una buena caracterización de los recursos humanos y sus competencias, así como una correcta selección de los roles necesarios y una buena asignación de responsabilidades para estos mismos. Por otro lado, la correcta comunicación del equipo fue algo visto a lo largo de todo el desarrollo, remarcando la correcta asignación de roles y tareas a los distintos miembros del equipo.

¿Qué se podía haber hecho mejor?

Ciertamente, tendimos a ser demasiado genéricos y teóricos en varias de las entregas, profundizando levemente en planteamientos importantes para el correcto desarrollo del proyecto, así como también quedó evidenciada la falta de coherencia entre entregables en distintas ocasiones.

¿Qué conocimientos aprendimos que podríamos aplicar a otros proyectos?

Labores importantes de la gestión de riesgo, como la identificación de riesgos y la elaboración de estrategias de mitigación y control de estos mismos, son enseñanzas importantes y aplicables no solo al desarrollo de cualquiera proyecto de software, sino también, desde nuestro punto de vista, aplicable a todo tipo de proyectos; así como también lo son elementos de la gestión de comunicaciones, como la matriz de comunicaciones y los procedimientos para mitigar inconvenientes, entre varias otras enseñanzas aprendidas a lo largo del desarrollo.

¿Qué recomendaciones haríamos?

Basándonos en la experiencia adquirida desarrollando este proyecto, recomendamos no limitarse a seguir exclusivamente las definiciones del PMBOK al pie de la letra, sino adaptar cada uno de los procesos al tamaño y complejidad que verdaderamente requiera el cliente; por otro lado, también recomendamos centralizar la información, asegurarse que todos los miembros tengan presente lo que se acordó en la última reunión, para que, de esta manera, ningún riesgo o requisito crítico quede olvidada en el camino.

IX. Aceptación del Cierre del Proyecto de Software

[Una vez concluido el proceso de cierre técnico y administrativo, el Gerente de Proyectos y el Patrocinador (Sponsor), deben firmar la aceptación del proyecto de software culminado.]

Aprobación del Cierre del Proyecto de Software	[Firma]
Líder del Proyecto:	Manuel Martinez
Patrocinador:	Agromarket
Fecha de Acta de Cierre:	11/01/2026
Lugar:	Agromarket HQ